

Lab 1: Basic Python Programming

CPE232 Data Models

[1] Variable

1.1 Number Variable

```
In [3]: num = 100 #integer variable
num2 = 12.5 #float variable
print(num)
print(num2)

print(num + num2) #addition
print(num - num2) #subtraction
print(num * num2) #multiplication
print( num / num2) #division
```

```
100
12.5
112.5
87.5
1250.0
8.0
```

1.2 String Variable

```
In [4]: string = "Data Models"
print(string) #print complete string

print("Hello " + string) #print concatenated string

# slicing string
print(string[0]) #print first character of the string
print(string[:4]) #print first to 4th character of the string
print(string[5:]) #print 6th to last character of the string
print(string[1:4]) #print 2nd to 4th character of the string
print(string * 2) #print string 2 times
```

```
Data Models
Hello Data Models
D
Data
Models
ata
Data ModelsData Models
```

```
In [5]: # format string
code = "CPE232"

print(f"{code}: Data Models")
```

```
CPE232: Data Models
```

1.3 Boolean Variable

```
In [6]: #boolean variable
boolean = True
boolean2 = False

print(boolean)           #print boolean variable
print(not boolean)       #print opposite of boolean variable
print(boolean and boolean2) #print boolean and boolean2
print(boolean or boolean2) #print boolean or boolean2
```

True
False
False
True

1.4 List Variable

```
In [7]: #list variable
list = ["Data", 20, 123.23, 40, 50]
another_list = ["Models", 60]

print(list)               #print complete list
print(list[0])            #print first element of the list
print(list[1:3])          #print 2nd to 3rd element of the list
print(list[2:])           #print 3rd to last element of the list
print(another_list)       #print complete another_list
print(another_list * 2)    #print another_list two times
print(list + another_list) #print concatenated list

list[0] = "CPE232"        #change first element of the list
print(list)               #print complete list

# add element at the end
list.append(True)
print(list)

# pop element by index
list.pop(1)
print(list)
```

```
['Data', 20, 123.23, 40, 50]
Data
[20, 123.23]
[123.23, 40, 50]
['Models', 60]
['Models', 60, 'Models', 60]
['Data', 20, 123.23, 40, 50, 'Models', 60]
['CPE232', 20, 123.23, 40, 50]
['CPE232', 20, 123.23, 40, 50, True]
['CPE232', 123.23, 40, 50, True]
```

1.5 Tuple Variable

```
In [8]: #tuple variable
tuple = ("Data", 20, 123.23, 40, 50)
another_tuple = ("Models", 60)
```

```

print(tuple)                #print complete tuple
print(tuple[0])              #print first element of the tuple
print(tuple[1:3])            #print 2nd to 3rd element of the tuple
print(tuple[2:])              #print 3rd to last element of the tuple
print(tuple * 2)              #print tuple two times
print(tuple + another_tuple) #print concatenated tuple

```

```

('Data', 20, 123.23, 40, 50)
Data
(20, 123.23)
(123.23, 40, 50)
('Data', 20, 123.23, 40, 50, 'Data', 20, 123.23, 40, 50)
('Data', 20, 123.23, 40, 50, 'Models', 60)

```

In [9]: `tuple[0] = "CPE232"` *# tuple is immutable dtype, we can't change its val*

```

-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 tuple[0] = "CPE232"          # tuple is immutable dtype, we can't change
its value after declared

TypeError: 'tuple' object does not support item assignment

```

1.6 Dictionary Variable

```

In [ ]: #dictionary variable
dictionary = {"name": "Alice", "age": 21}
another_dictionary = {}
another_dictionary["name"] = "Bob"
another_dictionary["age"] = 21

print(dictionary)                #print complete dictionary
print(dictionary["name"])         #print value for specific key
print(dictionary.keys())          #print all the keys
print(dictionary.values())        #print all the values
print(dictionary.items())         #print all the items
print(another_dictionary)         #print complete another_dictionary

```

```

{'name': 'Alice', 'age': 21}
Alice
dict_keys(['name', 'age'])
dict_values(['Alice', 21])
dict_items([('name', 'Alice'), ('age', 21)])
{'name': 'Bob', 'age': 21}

```

1.7 Set Variable

```

In [ ]: #set variable
my_set = {"Data", 20, 123.23, 60, "Data"}
another_set = {"Models", 60}

print(my_set)                    #print complete set (duplicates are

#set operations
print(my_set.union(another_set))  # print all unique elements from bot
print(my_set.intersection(another_set)) # print common elements from both se

```

```
# add element
my_set.add("New Item")
my_set.add(20)                                     # try adding a duplicate again
print(my_set)

# remove element
my_set.remove(123.23)
print(my_set)
```

```
{'Data', 123.23, 20, 60}
{'Data', 20, 'Models', 123.23, 60}
{60}
{'Data', 'New Item', 20, 123.23, 60}
{'Data', 'New Item', 20, 60}
```

[2] Control Flow

2.1 IF ... ELIF ... ELSE

```
In [ ]: number = 123
        number2 = 34

        if number > number2:
            print("number is greater than number2")
        elif number < number2:
            print("number is less than number2")
        else:
            print("number is equal to number2")
```

number is greater than number2

2.2 try ... except

```
In [ ]: number1 = "number"
        number2 = 2

        try:
            number1 + number2
        except:                                     #error handling
            print("What are you doing?")
```

What are you doing?

[3] Loop

3.1 For Loop

```
In [ ]: #for Loops
        for num in range(0,10):
            print(num)
```

0
1
2
3
4
5
6
7
8
9

```
In [ ]: #for Loops with step  
for num in range(5,15,2):  
    print(num)
```

5
7
9
11
13

```
In [ ]: #for Loop with enumerate  
  
list = ["Alice","Bob","Charlie","Daisy"]  
  
for index, name in enumerate(list):  
    print(f"{index}: {name}")
```

0: Alice
1: Bob
2: Charlie
3: Daisy

```
In [ ]: #for Loop with List  
  
for name in list:  
    print(name)
```

Alice
Bob
Charlie
Daisy

```
In [ ]: #continue in for loop  
  
list = [1,23,7,"hello",True,1123,43,23,12]  
  
for element in list:  
    if type(element) != int:  
        continue  
    print(element)
```

1
23
7
1123
43
23
12

```
In [ ]: #break in for Loop
```

```
list = [1,23,7,"hello",True,1123,43,23,12]

for element in list:
    if type(element) != int:
        break
    print(element)
```

1
23
7

3.2 While loop

```
In [ ]: #while Loop

list = ["Alice","Bob","Charlie","Daisy"]
count = 0

while count < len(list):
    print(list[count])
    count += 1
```

Alice
Bob
Charlie
Daisy

```
In [ ]: #continue in while Loop

list = [1,23,7,"hello",True,1123,43,23,12]
count = 0

while count < len(list):
    if type(list[count]) != int:
        count += 1
        continue
    print(list[count])
    count += 1
```

1
23
7
1123
43
23
12

```
In [ ]: #break in while Loop

list = [1,23,7,"hello",True,1123,43,23,12]
count = 0

while count < len(list):
    if type(list[count]) != int:
        break
    print(list[count])
    count += 1
```

1
23
7

[4] Function

```
In [ ]: #define function
def function_name (arg1, arg2):
    return arg1 + arg2

#calling function
function_name(1,2)
```

Out[]: 3

```
In [ ]: #define function with default argument
def function_with_default_arg(arg1, arg2 = 10, arg3 = 20 , arg4 = 30):
    return arg1 + arg2 + arg3 + arg4

result_1 = function_with_default_arg(1)
result_2 = function_with_default_arg(1,2,5)
result_3 = function_with_default_arg(1,2,5,10)

print(result_1)
print(result_2)
print(result_3)
```

61
38
18

```
In [ ]: #multiple arguments
def function_with_multiple_arg(*args):
    print(args)
    print(type(args))
    sum = 0
    for num in args:
        sum += num

    return sum

function_with_multiple_arg(1,2,3,4,5)
```

(1, 2, 3, 4, 5)
<class 'tuple'>

Out[]: 15

```
In [ ]: #lambda function
lambda_function = lambda arg1, arg2: arg1 + arg2

print(lambda_function(1,2))
```

3

[5] OOP

```
In [ ]: class Car:
        """A simple attempt to represent a car."""

        def __init__(self, make, model, year):
            """Initialize attributes to describe a car."""
            self.make = make
            self.model = model
            self.year = year
            self.odometer_reading = 0

        def get_description_name(self):
            """Return a neatly formatted descriptive name."""
            long_name = f"{self.year} {self.make} {self.model}"
            return long_name.title()

        def read_odometer(self):
            """Print a statement showing the car's mileage."""
            print(f"This car has {self.odometer_reading} miles on it.")

        def update_odometer(self, mileage):
            """Set the odometer reading to the given value."""
            self.odometer_reading = mileage
```

```
In [ ]: my_car = Car("Honda", "Civic", 2016)
        my_car.read_odometer()

        # update odometer
        my_car.update_odometer(50)
        my_car.read_odometer()
```

This car has 0 miles on it.

This car has 50 miles on it.

[6] File Handling

6.1 Text File

```
In [ ]: # write file
        with open("test.txt", "w") as file:
            file.write("Hello World")

        # read file
        with open("test.txt", "r") as file:
            print(file.read())
```

Hello World

6.2 CSV File

```
In [ ]: import csv

        with open("test.csv", "w", newline='') as file:
            writer = csv.writer(file)
            writer.writerow(["Name", "Surname"])
            writer.writerow(["Alice", "Johnson"])
            writer.writerow(["Bob", "Smith"])
```

```
In [ ]: import csv

with open("test.csv","r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

```
['Name', 'Surname']
['Alice', 'Johnson']
['Bob', 'Smith']
```

6.3 JSON file

```
In [ ]: import json

data = {
    "username": "admin",
    "password": "admin",
    "role": "sleeping",
}

with open('test.json', 'w') as f:
    json.dump(data, f)
```

```
In [ ]: with open("test.json", "r") as f:
        data = json.load(f)

data
```

```
Out[ ]: {'username': 'admin', 'password': 'admin', 'role': 'sleeping'}
```

[7] Libraries

7.1 Numpy

import numpy library

```
In [11]: %pip install numpy
import numpy as np
```

Requirement already satisfied: numpy in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (2.4.1)

Note: you may need to restart the kernel to use updated packages.

ndarray initialization

Construct using python list

```
In [172... # 1d ndarray from 1d python list
list_a1=[1,2,3.5]
arr_a1=np.array(list_a1)
arr_a1
```

```
Out[172... array([1. , 2. , 3.5])
```

```
In [13]: # 2d ndarray from 2d python list (list of list)
list_a2=[[1,2],[3,4],[5,6]]
arr_a2=np.array(list_a2)
arr_a2
```

```
Out[13]: array([[1, 2],
               [3, 4],
               [5, 6]])
```

```
In [14]: list_a3=[ [[1,2],[2,3]], [[3,4],[4,5]] ]
arr_a3=np.array(list_a3)
arr_a3
```

```
Out[14]: array([[[1, 2],
                 [2, 3]],

                [[3, 4],
                 [4, 5]]])
```

or construct using some numpy classes and functions

```
In [15]: np.zeros(5)
```

```
Out[15]: array([0., 0., 0., 0., 0.])
```

```
In [16]: np.ones((3,4),dtype=float)
```

```
Out[16]: array([[1., 1., 1., 1.],
               [1., 1., 1., 1.],
               [1., 1., 1., 1.]])
```

```
In [17]: np.eye(3, 3)
```

```
Out[17]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

```
In [18]: np.full((4,),999)
```

```
Out[18]: array([999, 999, 999, 999])
```

```
In [19]: np.arange(3,10,2)
```

```
Out[19]: array([3, 5, 7, 9])
```

```
In [20]: np.linspace(0, 1, 5)
```

```
Out[20]: array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

```
In [21]: np.random.choice(['a','b'], 9)
```

```
Out[21]: array(['a', 'b', 'a', 'b', 'b', 'a', 'b', 'b', 'a'], dtype='<U1')
```

```
In [22]: np.random.randn(10)
```

```
Out[22]: array([ 1.05722529, -1.30395375,  0.64071357,  0.89152784, -1.9461776 ,
                -1.12472831,  1.78123632,  0.40012828,  0.65449476,  0.30295749])
```

ndarray properties

```
In [174... list_a=[[1,2,3,4],[5,6,7,8],[9,10,11,12]]
arr_a=np.array(list_a)
arr_a
```

```
Out[174... array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

```
In [175... print("dimension:", arr_a.ndim)
print("shape:", arr_a.shape)
print("data type:", arr_a.dtype)
print("array's size (elements)", arr_a.size)
```

```
dimension: 2
shape: (3, 4)
data type: int64
array's size (elements) 12
```

Reshaping & Modification

from this original ndarray

```
In [176... arr_a
```

```
Out[176... array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

try to convert into 3D array

```
In [177... arr_a.reshape((2,2,3))
```

```
Out[177... array([[[ 1,  2,  3],
        [ 4,  5,  6]],

       [[ 7,  8,  9],
        [10, 11, 12]])]
```

sometimes you may resize for same dimension where only known some dimension,
insert -1 for unknown len

```
In [178... arr_a.reshape((-1,6))
```

```
Out[178... array([[ 1,  2,  3,  4,  5,  6],
        [ 7,  8,  9, 10, 11, 12]])
```

Would you like to try this?

```
In [179... arr_a.reshape((-1,5))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[179], line 1
----> 1 arr_a.reshape((-1,5))

ValueError: cannot reshape array of size 12 into shape (5)
```

[Q1] From the above cell, explain in your own words why it worked or did not work.

Ans: ไม่สามารถทำได้ เนื่องจาก arr_a ไม่สามารถแบ่งเป็น 5 columns ได้ โดยที่ในแต่ละ rows มีสมาชิกเท่ากัน ถ้าจะให้มันได้ต้องแบ่งจำนวน columns เป็นเลขที่หาร 12 ลงตัว

Next, try to append any value(s) into existing 2darray

```
In [180...] np.append(arr_a, 13)
```

```
Out[180...] array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13])
```

```
In [181...] np.append(arr_a, arr_a[0])
```

```
Out[181...] array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12,  1,  2,  3,  4])
```

```
In [182...] np.append(arr_a, arr_a[0].reshape((1,-1)),axis=0)
```

```
Out[182...] array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12],
        [ 1,  2,  3,  4]])
```

```
In [183...] np.append(arr_a, arr_a[:,0].reshape((-1,1)),axis=1)
```

```
Out[183...] array([[ 1,  2,  3,  4,  1],
        [ 5,  6,  7,  8,  5],
        [ 9, 10, 11, 12,  9]])
```

```
In [184...] np.concatenate([arr_a, arr_a])
```

```
Out[184...] array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12],
        [ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

```
In [185...] np.concatenate([arr_a, arr_a],axis=1)
```

```
Out[185...] array([[ 1,  2,  3,  4,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  5,  6,  7,  8],
        [ 9, 10, 11, 12,  9, 10, 11, 12]])
```

indexing & slicing

from this original array again

```
In [186...] arr_a
```

```
Out[186...] array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

try to access all element at the first row

```
In [187...] arr_a[1]
```

```
Out[187...] array([5, 6, 7, 8])
```

then you would like to access the second element from the first row

```
In [188...] arr_a[1][2]
arr_a[1,2]
```

```
Out[188...] np.int64(7)
```

Next, try to access all element start from the 2nd element in the first row

```
In [189...] arr_a[1,1:]
```

```
Out[189...] array([6, 7, 8])
```

```
In [190...] arr_a[:2, 1:]
```

```
Out[190...] array([[2, 3, 4],
        [6, 7, 8]])
```

sometimes you may specify some row number using list within indexing

```
In [191...] arr_a[[1,2,1], 1:]
```

```
Out[191...] array([[ 6,  7,  8],
        [10, 11, 12],
        [ 6,  7,  8]])
```

Boolean slicing

based on this original array

```
In [192...] arr_a
```

```
Out[192...] array([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]])
```

try to filter all elements which more than 5

```
In [193...] arr_a>5
```

```
Out[193...] array([[False, False, False, False],
        [False,  True,  True,  True],
        [ True,  True,  True,  True]])
```

Next, try to filter all elements which more than 5 and less than 10

```
In [194... (arr_a>5)&(arr_a<10)
```

```
Out[194... array([[False, False, False, False],
        [False, True, True, True],
        [ True, False, False, False]])
```

Run the cell below and answer a question.

```
In [195... arr_a[(arr_a>5)&(arr_a<10)]
```

```
Out[195... array([6, 7, 8, 9])
```

[Q2] From the above cell, explain in your own words how the output came about?

Ans: Output ที่ออกมาคือ ตัวเลขที่ Intersection กันระหว่างเซตที่มากกว่า 5 และ น้อยกว่า 10 หรือจะมองอีกแบบคือ เป็นการเทียบตัวเลขทีละตัวว่ามากกว่า 5 และ น้อยกว่า 10 หรือไม่ ถ้าเป็นจริงจะพิมพ์ออกมา

Try running the cell below.

```
In [196... arr_a[(arr_a>5) and (arr_a<10)]
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[196], line 1
----> 1 arr_a[(arr_a>5) and (arr_a<10)]

ValueError: The truth value of an array with more than one element is ambiguous.
Use a.any() or a.all()
```

[Q3] Explain in your own words why the above cell gives an error.

Ans: ที่ error เพราะ and เป็นการเทียบทั้ง Array เลย แต่ใน Array มีหลาย element จึงไม่สามารถนำมาเทียบได้ แต่ถ้าใช้ & เป็น operator จะเป็นการนำมาเทียบทีละตัว

[Q4] And what should be written instead so that the code is error-free?

Ans: เปลี่ยนจาก and เป็น & คือ arr_a[(arr_a > 5) & (arr_a < 10)]

Basic operations

```
In [208... list_b=[[1,2,3,4],[1,2,3,4],[1,2,3,4]]
arr_b=np.array(list_b)
arr_b
```

```
Out[208... array([[1, 2, 3, 4],
        [1, 2, 3, 4],
        [1, 2, 3, 4]])
```

This is some operations for only 1 array

```
In [209... np.sqrt(arr_b)
```

```
Out[209...] array([[1.          , 1.41421356, 1.73205081, 2.          ],
        [1.          , 1.41421356, 1.73205081, 2.          ],
        [1.          , 1.41421356, 1.73205081, 2.          ]])
```

This is some operations for 2 arrays with the same shape

```
In [210...] arr_a-arr_b
```

```
Out[210...] array([[0, 0, 0, 0],
        [4, 4, 4, 4],
        [8, 8, 8, 8]])
```

```
In [211...] np.add(arr_a,arr_b)
```

```
Out[211...] array([[ 2,  4,  6,  8],
        [ 6,  8, 10, 12],
        [10, 12, 14, 16]])
```

Next, try to operate with 1 array and one numeric variable

```
In [212...] arr_a*3
```

```
Out[212...] array([[ 3,  6,  9, 12],
        [15, 18, 21, 24],
        [27, 30, 33, 36]])
```

```
In [213...] 1+arr_a**2
```

```
Out[213...] array([[ 2,  5, 10, 17],
        [ 26, 37, 50, 65],
        [ 82, 101, 122, 145]])
```

Broadcasting

```
In [214...] arr_c=np.array([1,2,3])
arr_d=np.array([[3],[5],[8]])
```

```
In [215...] arr_c-arr_d
```

```
Out[215...] array([[ -2, -1,  0],
        [ -4, -3, -2],
        [ -7, -6, -5]])
```

Basic aggregations

```
In [216...] arr_a

print(arr_a)
print("sum:", arr_a.sum())
print("mean:", arr_a.mean())
print("min:", arr_a.min())
print("max:", arr_a.max())
print("standard deviation:", arr_a.std())
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
sum: 78
mean: 6.5
min: 1
max: 12
standard deviation: 3.452052529534663
```

ndarray axis

```
In [217... # column axis
arr_a.sum(axis=0)
```

```
Out[217... array([15, 18, 21, 24])
```

```
In [218... # row axis
arr_a.sum(axis=1)
```

```
Out[218... array([10, 26, 42])
```

[Q5] Summarize the value of the argument *axis*, what is the value for row-wise summation and column-wise summation, respectively?

Ans: Array ตัวอย่าง คือ [1 2 3 4] [5 6 7 8] [9 10 11 12] axis = 1 คือ แกวเป็นการบวกเลขของทั้งแถว เช่น แกวที่ 1 ผลบวกคือ $1 + 2 + 3 + 4 = 10$ ซึ่งคือ ผลรวมของแถวแรก axis = 0 คือ หลักเป็นการบวกเลขทั้งหลัก เช่น หลักที่ 1 ผลบวกคือ $1 + 5 + 9 = 15$ ซึ่งคือ ผลรวมของหลักแรก

7.2 Pandas

Series

```
In [219... %pip install pandas
import pandas as pd
import numpy as np
```

```
Requirement already satisfied: pandas in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from pandas) (2.4.1)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [220... pd.Series(np.random.randn(6))
```

```
Out[220...] 0    0.831758
            1   -0.576298
            2    0.767423
            3    0.555864
            4   -0.062981
            5    1.421770
            dtype: float64
```

```
In [221...] pd.Series(np.random.randn(6), index=['a','b','c','d','e','f'])
```

```
Out[221...] a    0.354842
            b   -0.129313
            c   -1.497792
            d    1.191249
            e   -0.643312
            f    0.089230
            dtype: float64
```

Constructing Dataframe

Constructing DataFrame from a dictionary

```
In [222...] d = {'col1':[1,2], 'col2': [3,4]}
```

```
In [223...] df = pd.DataFrame(data=d)
df
```

```
Out[223...]   col1  col2
0         1     3
1         2     4
```

```
In [224...] d2 = {'Name':['Joe','Nat','Harry','Sam','Monica'],
                  'Age': [20,21,19,20,22]}

df2 = pd.DataFrame(data=d2)
df2
```

```
Out[224...]   Name  Age
0      Joe   20
1      Nat   21
2    Harry   19
3      Sam   20
4  Monica   22
```

Constructing DataFrame from a List

```
In [225...] marks_list = [85.10, 77.80, 91.54, 88.78, 60.55]

df3 = pd.DataFrame(marks_list, columns=['Marks'])
df3
```

Out[225...

Marks**0** 85.10**1** 77.80**2** 91.54**3** 88.78**4** 60.55

Creating DataFrame from file

In [226...

```
# Read csv file from path and store to df for create dataframe
df = pd.read_csv('nss15.csv')
df
```

Out[226...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	
1	150734723	7/6/2015	83.2157	S	36	Male	White	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	
...	...	...	...	...	...	...	...	...
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	
334835	150733393	7/11/2015	5.6748	C	3	Female	Black	
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	
334837	150823002	8/8/2015	97.9239	M	38	Female	White	
334838	150723074	6/20/2015	49.2646	M	5	Female	White	

334839 rows × 12 columns

**Viewing DataFrame information**

(.shape, .head, .tail, .info, select column, .unique, .describe, select row with .loc and .iloc)

Check simple information

In [227...

```
# Check dimension by .shape
df.shape
```

Out[227...

(334839, 12)

In [228...

```
# Display the first 5 rows by default
df.head()
```

Out[228...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis	b
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	57	
1	150734723	7/6/2015	83.2157	S	36	Male	White	57	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	71	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	71	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	62	

In [229...

```
# Display the first 3 rows
df.head(3)
```

Out[229...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis	b
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	57	
1	150734723	7/6/2015	83.2157	S	36	Male	White	57	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	71	

In [230...

```
# Display the last 5 rows by default
df.tail()
```

Out[230...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	
334835	150733393	7/11/2015	5.6748	C	3	Female	Black	
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	
334837	150823002	8/8/2015	97.9239	M	38	Female	White	
334838	150723074	6/20/2015	49.2646	M	5	Female	White	

In [231...

```
# Overview information of dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 334839 entries, 0 to 334838
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   caseNumber            334839 non-null  int64
1   treatmentDate         334839 non-null  object
2   statWeight            334839 non-null  float64
3   stratum               334839 non-null  object
4   age                   334839 non-null  int64
5   sex                   334837 non-null  object
6   race                  205014 non-null  object
7   diagnosis             334839 non-null  int64
8   bodyPart              334839 non-null  int64
9   disposition           334839 non-null  int64
10  location              334839 non-null  int64
11  product               334839 non-null  int64
dtypes: float64(1), int64(7), object(4)
memory usage: 30.7+ MB
```

Select column, multiple column, with condition

In [232... `df.columns`

Out[232... `Index(['caseNumber', 'treatmentDate', 'statWeight', 'stratum', 'age', 'sex', 'race', 'diagnosis', 'bodyPart', 'disposition', 'location', 'product'], dtype='object')`

In [233... `# column slicing`
`df['age']`

Out[233... `0 5`
`1 36`
`2 20`
`3 61`
`4 88`
`..`
`334834 7`
`334835 3`
`334836 38`
`334837 38`
`334838 5`
`Name: age, Length: 334839, dtype: int64`

In [234... `df.age`

Out[234... `0 5`
`1 36`
`2 20`
`3 61`
`4 88`
`..`
`334834 7`
`334835 3`
`334836 38`
`334837 38`
`334838 5`
`Name: age, Length: 334839, dtype: int64`

Viewing the unique value

```
In [235... df.race.unique()
```

```
Out[235... array([nan, 'White', 'Other', 'Black', 'Asian', 'American Indian'],  
      dtype=object)
```

Null values

```
In [236... df.isnull().sum()
```

```
Out[236... caseNumber          0  
treatmentDate         0  
statWeight            0  
stratum              0  
age                  0  
sex                  2  
race                129825  
diagnosis            0  
bodyPart            0  
disposition         0  
location            0  
product            0  
dtype: int64
```

Describe

```
In [237... df['age'].describe()
```

```
Out[237... count    334839.000000  
mean       31.385451  
std        26.105098  
min         0.000000  
25%        10.000000  
50%        23.000000  
75%        51.000000  
max       107.000000  
Name: age, dtype: float64
```

Slicing dataframe

```
In [238... # select multiple column  
df[['treatmentDate', 'statWeight', 'age', 'sex']]
```

Out[238...

	treatmentDate	statWeight	age	sex
0	7/11/2015	15.7762	5	Male
1	7/6/2015	83.2157	36	Male
2	8/2/2015	74.8813	20	Female
3	6/26/2015	15.7762	61	Male
4	7/4/2015	74.8813	88	Female
...	...	...	...	...
334834	5/31/2015	15.0591	7	Male
334835	7/11/2015	5.6748	3	Female
334836	7/24/2015	15.7762	38	Male
334837	8/8/2015	97.9239	38	Female
334838	6/20/2015	49.2646	5	Female

334839 rows × 4 columns

In [239...

```
#select by condition
df[df['sex'] == 'Male']
```

Out[239...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	57
1	150734723	7/6/2015	83.2157	S	36	Male	White	57
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	71
6	150713483	6/8/2015	15.7762	V	25	Male	Black	51
7	150704114	6/14/2015	83.2157	S	53	Male	White	57
...	...	...	...	...	...	...	...	..
334824	150607827	5/27/2015	5.6748	C	1	Male	White	71
334825	150600190	5/28/2015	80.8381	S	5	Male	NaN	56
334833	150747217	7/24/2015	83.2157	S	2	Male	NaN	62
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	59
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	71

182501 rows × 12 columns



In [240...

```
# select by multiple condition
df[(df['sex'] == 'Male') & (df['age'] > 80)]
```

Out[240...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis
8	150736558	7/16/2015	83.2157	S	98	Male	Black	59
63	150418623	1/12/2015	15.0591	V	97	Male	Other	62
97	150700375	6/28/2015	83.2157	S	85	Male	NaN	59
131	150940801	9/14/2015	15.7762	V	96	Male	NaN	62
177	160110774	12/19/2015	85.7374	S	81	Male	White	59
...	...	...	...	...	...	...	...	..
334616	160104368	12/30/2015	74.8813	L	86	Male	Other	71
334677	151115099	11/4/2015	16.5650	V	83	Male	NaN	63
334699	150633387	5/29/2015	74.8813	L	84	Male	NaN	53
334701	150515945	4/27/2015	97.9239	M	86	Male	NaN	57
334785	150733286	7/11/2015	15.7762	V	86	Male	White	71

6379 rows × 12 columns



Select row with .iloc

In [241...

```
# row slicing
df.iloc[10:15]
```

Out[241...

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis
10	150734952	7/4/2015	15.7762	V	20	Male	Black	59
11	150821622	7/20/2015	83.2157	S	20	Female	White	57
12	150713631	7/4/2015	15.7762	V	11	Male	NaN	60
13	150666343	6/27/2015	15.7762	V	26	Female	White	62
14	150748843	7/16/2015	37.6645	L	33	Male	Asian	53



In [242...

```
# column slicing
df.iloc[:, [0,1,2,3,4]]
```

Out[242...

	caseNumber	treatmentDate	statWeight	stratum	age
0	150733174	7/11/2015	15.7762	V	5
1	150734723	7/6/2015	83.2157	S	36
2	150817487	8/2/2015	74.8813	L	20
3	150717776	6/26/2015	15.7762	V	61
4	150721694	7/4/2015	74.8813	L	88
...	...	...	...	...	...
334834	150739278	5/31/2015	15.0591	V	7
334835	150733393	7/11/2015	5.6748	C	3
334836	150819286	7/24/2015	15.7762	V	38
334837	150823002	8/8/2015	97.9239	M	38
334838	150723074	6/20/2015	49.2646	M	5

334839 rows × 5 columns

Select column and row with .loc

In [262...

```
# select column and row by .loc
df.loc[:6, 'treatmentDate': 'diagnosis']
```

Out[262...

	treatmentDate	statWeight	stratum	age	sex	race	diagnosis
0	7/11/2015	15.7762	V	5	Male	NaN	57
1	7/6/2015	83.2157	S	36	Male	White	57
2	8/2/2015	74.8813	L	20	Female	NaN	71
3	6/26/2015	15.7762	V	61	Male	NaN	71
4	7/4/2015	74.8813	L	88	Female	Other	62
5	7/2/2015	5.6748	C	1	Female	White	71
6	6/8/2015	15.7762	V	25	Male	Black	51

In [270...

```
# select row by condition
df.loc[df['age'] > 80, ['treatmentDate', 'age']]
```

Out[270...

	treatmentDate	age
4	7/4/2015	88
8	7/16/2015	98
39	5/3/2015	88
46	4/15/2015	91
63	1/12/2015	97
...	...	...
334701	4/27/2015	86
334784	7/7/2015	82
334785	7/11/2015	86
334815	10/28/2015	85
334819	1/13/2015	85

20422 rows × 2 columns

[Q6] What is the difference between .iloc and .loc?

Ans: .iloc จะกำหนด แถว หรือ หลัก ที่จะพิมพ์ผ่านการบอกเป็นตำแหน่ง เช่น df.iloc[0:4,0:4] เป็นการเลือกแถวที่ 0 ถึง 3 และ หลักที่ 0 ถึง 3 แต่ .loc จะเป็นการกำหนด แถว หรือ หลัก ผ่านการบอกเป็นชื่อ เช่น df.loc[0:4,'caseNumber':'stratum'] เป็นการเลือกแถวที่ชื่อ 0 ถึง 3 และ หลักที่ชื่อ caseNumber ถึง stratum

Basic aggregations

In [291...

```
# count elements in column via pandas Series method
df['sex'].value_counts()
```

Out[291...

```
sex
Male    182501
Female  152336
Name: count, dtype: int64
```

In [292...

```
# count elements in column via pandas function
pd.crosstab(df['sex'], columns='N')
```

Out[292...

col_0	N
sex	
Female	152336
Male	182501

In [309...

```
summary_by_sex = df.groupby('sex').agg({
    'age': 'mean',
    'statWeight': 'sum',
    'caseNumber': 'count',
```

```
'stratum': lambda x: x.mode()[0]
}).rename(columns={'caseNumber': 'total_cases'})

print(summary_by_sex)
```

	age	statWeight	total_cases	stratum
sex				
Female	35.585699	6.208773e+06	152336	V
Male	27.879792	6.964776e+06	182501	V

```
In [299... df['age'].apply('mean')
```

```
Out[299... np.float64(31.38545091820248)
```

```
In [307... df['race'].apply(len)
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[307], line 1
----> 1 df[ ].apply(len)

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\Week1\.venv\Lib\site-packa
ges\pandas\core\series.py:4943, in Series.apply(self, func, convert_dtype, args,
by_row, **kwargs)
    4808 def apply(
    4809     self,
    4810     func: AggFuncType,
    (...) 4815     **kwargs,
    4816 ) -> DataFrame | Series:
    4817     """
    4818     Invoke function on values of Series.
    4819
    (...) 4934     dtype: float64
    4935     """
    4936     return SeriesApply(
    4937         self,
    4938         func,
    4939         convert_dtype=convert_dtype,
    4940         by_row=by_row,
    4941         args=args,
    4942         kwargs=kwargs,
-> 4943     ).apply()

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\Week1\.venv\Lib\site-packa
ges\pandas\core\apply.py:1422, in SeriesApply.apply(self)
    1419     return self.apply_compat()
    1421 # self.func is Callable
-> 1422 return self.apply_standard()

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\Week1\.venv\Lib\site-packa
ges\pandas\core\apply.py:1502, in SeriesApply.apply_standard(self)
    1496 # row-wise access
    1497 # apply doesn't have a `na_action` keyword and for backward compat reason
s
    1498 # we need to give `na_action="ignore"` for categorical data.
    1499 # TODO: remove the `na_action="ignore"` when that default has been change
d in
    1500 # Categorical (GH51645).
    1501 action = "ignore" if isinstance(obj.dtype, CategoricalDtype) else None
-> 1502 mapped = obj._map_values(
    1503     mapper=curried, na_action=action, convert=self.convert_dtype
    1504 )
    1506 if len(mapped) and isinstance(mapped[0], ABCSeries):
    1507     # GH#43986 Need to do list(mapped) in order to get treated as nested
    1508     # See also GH#25959 regarding EA support
    1509     return obj._constructor_expanddim(list(mapped), index=obj.index)

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\Week1\.venv\Lib\site-packa
ges\pandas\core\base.py:925, in IndexOpsMixin._map_values(self, mapper, na_actio
n, convert)
    922 if isinstance(arr, ExtensionArray):
    923     return arr.map(mapper, na_action=na_action)
--> 925 return algorithms.map_array(arr, mapper, na_action=na_action, convert=con
vert)

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\Week1\.venv\Lib\site-packa

```

```

ges\pandas\core\algorithms.py:1743, in map_array(arr, mapper, na_action, convert)
    1741 values = arr.astype(object, copy=False)
    1742 if na_action is None:
-> 1743     return lib.map_infer(values, mapper, convert=convert)
    1744 else:
    1745     return lib.map_infer_mask(
    1746         values, mapper, mask=isna(values).view(np.uint8), convert=convert
    1747     )

```

File pandas/_libs/lib.pyx:2999, in pandas._libs.lib.map_infer()

TypeError: object of type 'float' has no len()

[Q7] In the above cell, Explain why pandas .agg()/apply() method accepts 'len' and 'lambda' without quotation marks.

Ans: เพราะว่า len และ lambda ไม่มีอยู่ใน pandas จึงต้องเรียกผ่าน Default python ถ้าการมี quotation marks จะเป็นการเรียกผ่าน pandas

7.3 Matplotlib

```

In [ ]: %pip install matplotlib
import matplotlib.pyplot as plt
import numpy as np

```

Requirement already satisfied: matplotlib in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (2.4.1)
Requirement already satisfied: packaging>=20.0 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (12.1.0)
Requirement already satisfied: pyparsing>=3 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (3.3.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\lchan\onedrive\desktop\kmutt\y2t2\cpe232\week1\.venv\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.

```

In [ ]: x = np.linspace(0, 10, 100)
        y = np.sin(x)

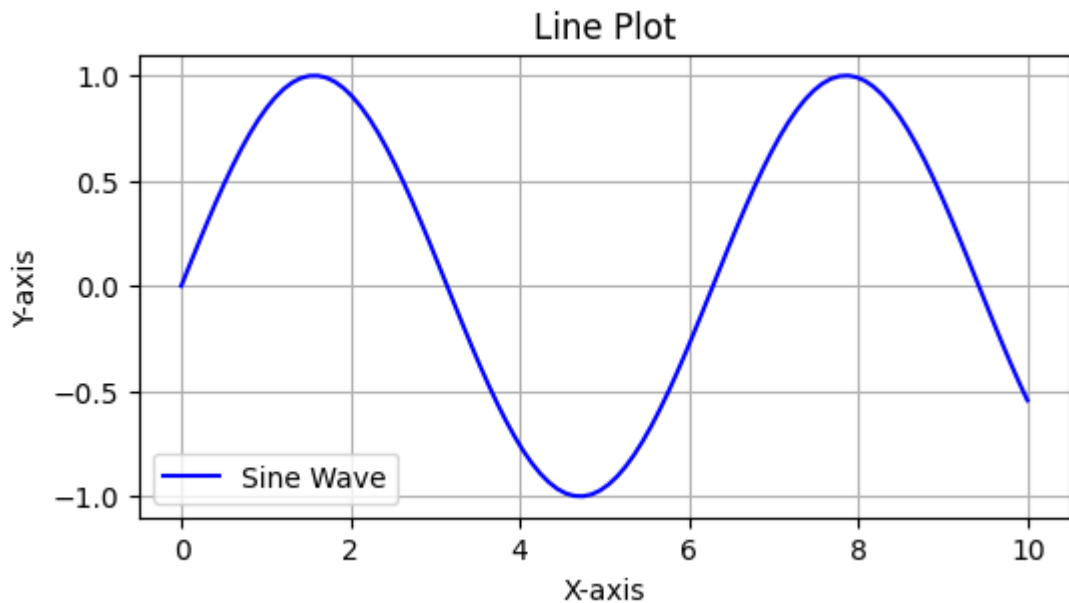
        # Line plot

        plt.figure(figsize=(6, 3))
        plt.plot(x, y, label='Sine Wave', color='blue')

```

```
plt.title('Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)

plt.savefig("sine.png")
plt.show()
```

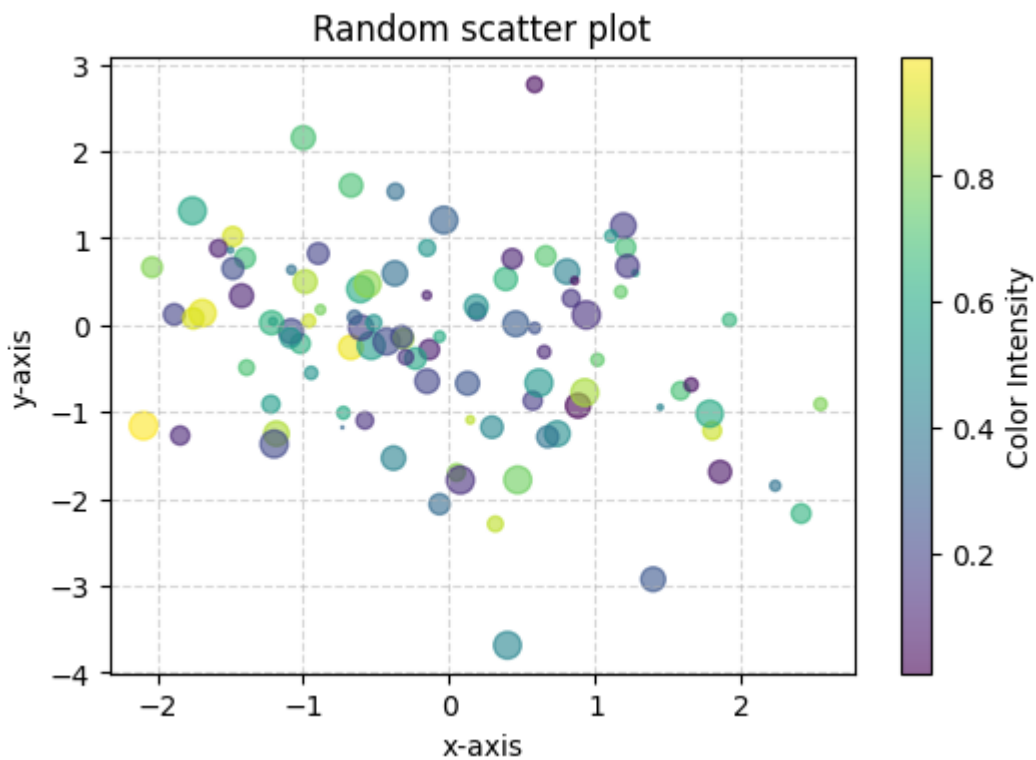


```
In [ ]: x = np.random.randn(100)
y = np.random.randn(100)
colors = np.random.rand(100) # Random colors
sizes = np.random.rand(100) * 100 # Random sizes

# Scatter plot
plt.figure(figsize=(6, 4))
scatter = plt.scatter(x, y, c=colors, s=sizes, alpha=0.6, cmap='viridis')

# Add styling and labels
plt.title('Random scatter plot')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.colorbar(scatter, label='Color Intensity')
plt.grid(True, linestyle='--', alpha=0.5)

# Show the plot
plt.show()
```



```
In [ ]: plt.figure(figsize=(5, 3))

# bar plot

df['sex'].value_counts().plot(kind='bar', color=['salmon', 'lightblue'])
plt.title('Gender Distribution')
plt.xlabel('Sex')
plt.ylabel('Count')
plt.xticks(rotation=0)
```

```
Out[ ]: (array([0, 1]), [Text(0, 0, 'Male'), Text(1, 0, 'Female')])
```

